Waseet — Backend Implementation Prompt

Oct 20, 2026 · @Osama Riyad

A paste-ready brief for a coding agent: hand it this whole document to implement the Waseet backend. It embeds the SRS, the approved architecture, the project structure, the tech stack, and the database design, plus the implementation rules and a per-feature checklist. Backend only — the frontend is a separate app that consumes this API.

Role & project

Act as a Senior Backend Engineer and implement the backend for the following software, using this document as the complete and primary brief.

Project — Waseet: an AI-mediated escrow service on PayPal. A buyer funds a deal through PayPal; the platform holds the funds in escrow; an AI verifies the seller's deliverable against agreed acceptance criteria and auto-releases payment on a confident pass, or routes the deal to a human-reviewed dispute. All money movement runs against the PayPal **sandbox** (no real funds). Implement the **backend only** — a separate frontend consumes this API.

SRS (embedded)

This is the condensed, authoritative requirement set. Treat it as the source of truth.

Functional requirements (Must = required; Should = strengthens; Could = stretch):

ID	Requirement	Priority
FR-1	Create a deal from a plain-language description (buyer, seller, amount, currency, description).	Must
FR-2	AI drafts a structured acceptance checklist; both parties confirm before funding.	Must
FR-3	Buyer funds the deal via PayPal; captured funds held by the platform account (state = FUNDED).	Must
FR-4	Seller submits a deliverable (URL, file, image, repo link, or text).	Must
FR-5	On submission, AI verifies against each criterion: per-criterion pass/fail, verdict, confidence, reasoning.	Must

ID	Requirement	Priority
FR-6	High-confidence pass auto-releases funds to the seller via Payout.	Must
FR-7	High-confidence fail, low confidence, or a buyer contest moves the deal to DISPUTED.	Must
FR-8	In DISPUTED, AI recommends release/refund/split; a human makes the final decision.	Must
FR-9	Refund returns captured funds to the buyer; split pays part to seller and refunds the rest.	Must / Could (split)
FR-10	Every state change and decision is written to an append-only audit log linked to the PayPal reference id.	Must
FR-11	Both parties can read a deal timeline: current state, verification report, dispute outcome.	Should
FR-12	Parties are updated on key events (funded, delivered, verified, released, refunded, disputed) via webhook-driven state.	Should
FR-13	The platform can take a configurable fee on release.	Could

Lifecycle states: DRAFT → FUNDED → DELIVERED → VERIFYING → RELEASED | REFUNDED | DISPUTED; DISPUTED → RELEASED / REFUNDED / SPLIT. RELEASED, REFUNDED and SPLIT are terminal. A central state machine owns the allowed transitions and guards; no service mutates state outside it.

PayPal (sandbox): Orders v2 (create + capture into the platform account = "escrow"); Payouts (release to seller); Payments v2 refund (refund to buyer); Webhooks (confirm capture/payout/refund, drive transitions). Three sandbox accounts: buyer (personal), seller (personal), platform (business). Optionally call these through the PayPal Agent Toolkit / MCP; otherwise wrap the REST API. Never release without a recorded AI or human decision.

AI components: criteria drafter (deal description → testable checklist); deliverable verifier (deliverable + criteria → per-criterion pass/fail + verdict + confidence + reasoning); decision policy (deterministic: confident pass → release, else → dispute; low confidence never auto-releases); dispute mediator (→ release/refund/split recommendation for the human). The verdict is advisory and always backed by the dispute path.

Non-functional: secrets in env only; verify webhook signatures; idempotent transitions and payouts (a replayed webhook never double-pays); verification returns in seconds with a visible VERIFYING state; append-only audit log; send the model only the data needed; present the AI verdict as advisory, not a guarantee.

Approved architecture

Layered (clean) architecture. Dependencies point inward: transport depends on services, services depend on repositories and gateway interfaces, domain depends on nothing.

Layer	Responsibility	Must not
API (routers + schemas)	HTTP, request/response validation, auth dependency, mapping to/from services	Contain business logic or touch the DB directly
Services / use cases	All business logic: deal, verification, settlement, dispute; orchestrate repos + gateways	Know about HTTP or SQLAlchemy sessions beyond what repos expose
Domain	Entities, enums, the escrow state machine (allowed transitions + guards)	Import framework, DB, or network code
Repositories	Data access per aggregate (deal, deliverable, verification, audit, webhook)	Contain business rules
Gateways	PayPal and AI behind interfaces (Protocols), so both are mockable in tests	Leak vendor types into services
Core	Config/secrets, structured logging, error handling, idempotency helpers	—

Boundary rules: business logic lives only in services and the domain; the state machine is the single place state changes are decided; PayPal and AI are reached only through their gateway interfaces; every external input is validated at the API boundary with Pydantic; money-moving operations run inside a DB transaction and are idempotent.

Project structure

```
waseet-backend/
  app/
    main.py                 # FastAPI app factory, router registration
    config.py               # Settings; secrets loaded from env (Pydantic
BaseSettings)
    database.py             # engine, session factory, Base
    dependencies.py         # auth + DB session dependencies
    api/
      routers/
        deals.py            # create deal, read deal, timeline
        funding.py          # create + capture order
        deliverables.py     # submit deliverable
        verification.py     # trigger / fetch verification
        disputes.py         # human decision (release/refund/split)
        webhooks.py         # PayPal webhook receiver
      schemas/              # Pydantic request/response models
    domain/
      entities.py           # domain models
      enums.py              # DealState, DeliverableType, Verdict, DecidedBy
      state_machine.py      # allowed transitions + guards
      decision_policy.py    # confident-pass -> release, else -> dispute
    services/
      deal_service.py
      verification_service.py
      settlement_service.py # release / refund / split
      dispute_service.py
    repositories/
      deal_repo.py
      deliverable_repo.py
      verification_repo.py
      audit_repo.py
      webhook_repo.py
    integrations/
      paypal/
        interface.py        # PayPalGateway protocol
        client.py           # Orders, Payouts, Refunds, Webhooks (httpx)
      ai/
        interface.py        # AIGateway protocol
        client.py           # criteria drafter, verifier, mediator
        prompts.py
    models/                 # SQLAlchemy ORM models
    core/
      errors.py             # exception types + handlers
      logging.py            # structlog setup
```

```
    idempotency.py
  migrations/               # Alembic
  tests/
    unit/
      test_state_machine.py
      test_decision_policy.py
      test_settlement_service.py
    integration/
      test_funding_flow.py
      test_verification_flow.py
      test_dispute_flow.py
    conftest.py             # fixtures; mocked PayPal + AI gateways
  pyproject.toml
  alembic.ini
  Dockerfile
  .env.example
  README.md
```

Tech stack

Concern	Choice
Language	Python 3.12
Web framework	FastAPI
ORM + migrations	SQLAlchemy 2.0 + Alembic
Validation	Pydantic v2 (request/response schemas + settings)
Database	PostgreSQL
HTTP client (PayPal)	httpx
AI	An LLM SDK (Anthropic or OpenAI) behind an AIGateway interface
Auth	JWT bearer tokens; role-based (buyer / seller / reviewer)
Tests	pytest + pytest-asyncio; PayPal and AI gateways mocked
Logging	structlog (structured, request-scoped)
Packaging / deploy	uv or Poetry; Docker; deployable to Render

Rationale: FastAPI + Pydantic give clean request validation and testable services; SQLAlchemy + a Postgres transaction model fit the money-movement idempotency requirements; both the PayPal and AI dependencies sit behind interfaces so the critical business logic can be unit-tested without network calls.

Database design

PostgreSQL. All ids are UUID primary keys; all tables carry created_at; money is stored as a decimal/numeric with an explicit currency, never a float.

Table	Columns	Notes
users	id, email (unique), role (enum: buyer/seller/reviewer), paypal_account_ref, created_at	Minimal identities for auth
deals	id, buyer_id (fk users), seller_id (fk users), description, amount (numeric), currency, platform_fee (numeric, nullable), state (enum deal_state), paypal_order_id, paypal_capture_id, payout_batch_id, created_at, updated_at	Central aggregate; state follows the state machine
acceptance_criteria	id, deal_id (fk), text, confirmed_by_buyer (bool), confirmed_by_seller (bool), created_at	One row per testable criterion
deliverables	id, deal_id (fk), type (enum deliverable_type), content_ref, submitted_at	What the seller submits
verification_reports	id, deal_id (fk), verdict (enum verdict), confidence (numeric), reasoning (text), per_criterion (jsonb), decided_by (enum decided_by), decided_at	One per verification or human decision
audit_log	id, deal_id (fk), from_state, to_state, actor, reason, paypal_reference, created_at	Append-only; no updates or deletes

Table	Columns	Notes
webhook_events	id, event_id (unique), event_type, payload (jsonb), processed_at (nullable)	Dedup key for idempotent webhook handling

Enums: `deal_state` = DRAFT, FUNDED, DELIVERED, VERIFYING, RELEASED, REFUNDED, DISPUTED, SPLIT · `deliverable_type` = url, file, image, repo, text · `verdict` = pass, fail, inconclusive · `decided_by` = ai, human · `role` = buyer, seller, reviewer.

Relationships & indexes: a deal has many acceptance_criteria, deliverables, verification_reports, and audit_log rows. Index deals(state) and deals(buyer_id, seller_id) for the timeline/list reads; unique index on webhook_events(event_id) enforces webhook idempotency; index audit_log(deal_id, created_at) for the timeline.

Implementation rules

1. Follow the SRS as the primary source of truth.
2. Follow the approved architecture and module boundaries.
3. Implement business logic separately from transport/framework concerns.
4. Implement validation at appropriate boundaries (Pydantic at the API edge; domain invariants in services).
5. Implement authentication and authorization correctly — only a deal's parties can act on it; only a reviewer can resolve a dispute.
6. Use consistent error handling (typed exceptions mapped to HTTP responses; no leaking stack traces).
7. Use transactions where required — every state change plus its money movement and audit entry commit atomically.
8. Prevent duplicated business logic — state transitions go through the single state machine; settlement logic lives in one service.
9. Validate all external input, including PayPal webhook payloads and signatures.
10. Never expose secrets or sensitive data — credentials from env only; never log tokens or full payloads.
11. Add logging and observability where appropriate (structured logs keyed by deal id and state).
12. Write tests for critical business logic (state machine, decision policy, settlement, webhook idempotency).
13. Use production-quality naming and code organization.

Features to implement

For every feature below: briefly explain the implementation; implement the models/entities; implement the services/use cases; implement repositories/data access; implement the API endpoints/controllers; implement validation; implement tests. Do not add features that are not required unless clearly marked optional.

#	Feature	Covers	Key endpoints
1	Deal creation + AI criteria drafting	FR-1, FR-2	POST /deals, POST /deals/{id}/criteria/confirm
2	Funding via PayPal (create + capture order into platform)	FR-3	POST /deals/{id}/order, POST /deals/{id}/capture
3	Deliverable submission	FR-4	POST /deals/{id}/deliverable
4	AI verification of the deliverable	FR-5	POST /deals/{id}/verify, GET /deals/{id}/verification
5	Decision policy + auto-release	FR-6, FR-7	(internal, triggered after verify)
6	Settlement: release / refund / split	FR-6, FR-9, FR-13	handled by settlement_service
7	Dispute + human decision	FR-7, FR-8	POST /deals/{id}/dispute/resolve
8	Append-only audit log	FR-10	GET /deals/{id}/audit
9	PayPal webhooks + idempotency	FR-12	POST /webhooks/paypal
10	Deal timeline / read	FR-11	GET /deals/{id}, GET /deals
11	Auth & authorization	NFR security	POST /auth/login + bearer dependency

The state machine and decision policy are the spine: feature 5 calls the decision policy, which asks the state machine for the next state, which feature 6 settles and feature 8 records — all in one transaction.

Final check

Before finalizing, compare the implementation against the SRS above and report:

- Any functional requirement (FR-1 to FR-13) not implemented, or only partially implemented.

- Any assumption made where the SRS was silent (for example: default confidence threshold for auto-release, fee percentage, token expiry).
- Any deviation from the approved architecture or module boundaries, with the reason.

Do not add features beyond the SRS unless they are marked optional (Should / Could). Deliver the code, the migrations, the tests, and a short README with setup and run instructions (PayPal sandbox credentials via environment variables) so it can be run end to end.